In [7]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error
from sklearn.preprocessing import StandardScaler
```

In [8]:
```
import pandas as pd
#housing = fetch_california_housing()
df = pd.read_csv('/home/backup/ml_datasets/california_housing.csv');
#df = pd.DataFrame(housing.data, columns=housing.feature_names)

#df['MedHouseVal']=housing.target

print(df.head(20))
```

```
    MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Latitude  \
0   8.3252      41.0  6.984127   1.023810       322.0  2.555556     37.88
1   8.3014      21.0  6.238137   0.971880      2401.0  2.109842     37.86
2   7.2574      52.0  8.288136   1.073446       496.0  2.802260     37.85
3   5.6431      52.0  5.817352   1.073059       558.0  2.547945     37.85
4   3.8462      52.0  6.281853   1.081081       565.0  2.181467     37.85
5   4.0368      52.0  4.761658   1.103627       413.0  2.139896     37.85
6   3.6591      52.0  4.931907   0.951362      1094.0  2.128405     37.84
7   3.1200      52.0  4.797527   1.061824      1157.0  1.788253     37.84
8   2.0804      42.0  4.294118   1.117647      1206.0  2.026891     37.84
9   3.6912      52.0  4.970588   0.990196      1551.0  2.172269     37.84
10  3.2031      52.0  5.477612   1.079602       910.0  2.263682     37.85
11  3.2705      52.0  4.772480   1.024523      1504.0  2.049046     37.85
12  3.0750      52.0  5.322650   1.012821      1098.0  2.346154     37.85
13  2.6736      52.0  4.000000   1.097701       345.0  1.982759     37.84
14  1.9167      52.0  4.262903   1.009677      1212.0  1.954839     37.85
15  2.1250      50.0  4.242424   1.071970       697.0  2.640152     37.85
16  2.7750      52.0  5.939577   1.048338       793.0  2.395770     37.85
17  2.1202      52.0  4.052805   0.966997       648.0  2.138614     37.85
18  1.9911      50.0  5.343675   1.085919       990.0  2.362768     37.84
19  2.6033      52.0  5.465455   1.083636       690.0  2.509091     37.84

    Longitude  MedHouseVal
0     -122.23        4.526
1     -122.22        3.585
2     -122.24        3.521
3     -122.25        3.413
4     -122.25        3.422
5     -122.25        2.697
6     -122.25        2.992
7     -122.25        2.414
8     -122.26        2.267
9     -122.25        2.611
10    -122.26        2.815
11    -122.26        2.418
12    -122.26        2.135
13    -122.26        1.913
14    -122.26        1.592
15    -122.26        1.400
16    -122.27        1.525
17    -122.27        1.555
18    -122.26        1.587
19    -122.27        1.629
```

In [9]:
```
Q1,Q3 = df ['AveRooms'].quantile([0.25,0.75])
IQR = Q3 - Q1
lower, upper = Q1 - 1.5*IQR, Q3 +1.5 *IQR
```

```
mask = (df['AveRooms'] >= lower) & (df['AveRooms'] <=upper)
df_clean = df.loc[mask].reset_index(drop=True)

print(f"Removed {len(df) - len(df_clean)} outlier rows; {len(df_clean)} remain")

X=df_clean[['AveRooms']].values
y=df_clean['MedHouseVal'].values
```

```
Removed 511 outlier rows; 20129 remain
```

In [10]:

```
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=42)

print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"y_test shape: {y_test.shape}")
```

```
X_train shape: (16103, 1)
X_test shape: (4026, 1)
y_train shape: (16103,)
y_test shape: (4026,)
```

In [11]:

```
#preprocessing
from sklearn.preprocessing import StandardScaler
scalar= StandardScaler()
scalar.fit(X_train)
X_train_scaled=scalar.transform(X_train)
X_test_scaled=scalar.transform(X_test)
print("Mean/std used for scaling:",scalar.mean_,scalar.scale_)
```

```
Mean/std used for scaling: [5.2445748] [1.15439852]
```

In [12]:

```
 def normal_equation(X,y):
        m= X.shape[0]
        X_b = np.c_[np.ones((m,1)),X]
        theta = np.linalg.inv(X_b.T @ X_b) @ X_b.T @ y
        return theta
```

In [31]:

```
theta_ne = normal_equation(X_train_scaled, y_train)
b_ne, w_ne = theta_ne[0], theta_ne[1]

print(f"Normal Equation parameters (on scaled feature): w = {w_ne:.4f}, b = {b_ne:.4
```

```
Normal Equation parameters (on scaled feature): w = 0.3662, b = 2.0581
```

In [14]:

```
def predict(X,w,b):
    return w*X.ravel() + b
y_pred_ne = predict(X_test_scaled, w_ne, b_ne)
print(y_pred_ne)
```

```
[1.89664984 1.55816522 1.91741673 ... 2.25792675 2.53992263 1.26072053]
```

In [15]:

```
mse = mean_squared_error(y_test, y_pred_ne)
print(f"Mean Squared Error: {mse}")
r2 = r2_score(y_test,y_pred_ne)
print(f"R-squared score: {r2}")
```

```
Mean Squared Error: 1.1851859323405964
R-squared score: 0.08456637521347243
```

In [32]:
```
x_line_scaled = np.linspace(X_test_scaled.min(), X_test_scaled.max(),200).reshape(-1
y_line = predict(x_line_scaled,w_ne, b_ne)
x_line_original = scalar.inverse_transform(x_line_scaled).ravel()

plt.figure(figsize=(8, 6))
plt.scatter(X_test, y_test, alpha=0.3, s=15, label="test data")
plt.plot(x_line_original, y_line, color = "red", linewidth =0.8)
plt.xlabel("Average Rooms per household")
plt.ylabel("Median house value")
plt.title("California Housing")
plt.show()
```



In [27]:
```
def compute_cost(X, y, w, b):
    m = len(y)
    predictions = w * X.ravel() + b
    cost = (1 / (2*m)) * np.sum((predictions- y) **2)
    return cost
```

In [28]:
```
def gradient(X, y, w_init= 0.0, b_init=0.0, alpha=0.01, num_iters=1000):
    m = len(y)
    w, b = w_init, b_init
    cost_history = []

    for i in range(num_iters):
        predictions = w* X.ravel() + b
        error = predictions - y

        dw = (1 / m) * np.dot(error, X.ravel())
        db = (1 / m) * np.sum(error)
```

```
        w -= alpha * dw
        b -= alpha * db

        cost = compute_cost(X, y, w, b)
        cost_history.append(cost)

        if i % 100 == 0:
            print(f"Iteration {i:4d}: cost = {cost:.5f}, w = {w:.4f}, b = {b:.4f}")

    return w, b, cost_history
```

In [29]:

```
w_gd, b_gd, cost_history = gradient(X_train_scaled, y_train, w_init=0.0, b_init=0.0,

print(f"\nFinal parameters (on scaled feature): w = {w_gd:.4f}, b = {b_gd:.4f}")
```

```
Iteration    0: cost = 2.34879, w = 0.0366, b = 0.2058
Iteration  100: cost = 0.57907, w = 0.3662, b = 2.0580
Iteration  200: cost = 0.57907, w = 0.3662, b = 2.0581
Iteration  300: cost = 0.57907, w = 0.3662, b = 2.0581
Iteration  400: cost = 0.57907, w = 0.3662, b = 2.0581
Iteration  500: cost = 0.57907, w = 0.3662, b = 2.0581
Iteration  600: cost = 0.57907, w = 0.3662, b = 2.0581
Iteration  700: cost = 0.57907, w = 0.3662, b = 2.0581
Iteration  800: cost = 0.57907, w = 0.3662, b = 2.0581
Iteration  900: cost = 0.57907, w = 0.3662, b = 2.0581

Final parameters (on scaled feature): w = 0.3662, b = 2.0581
```

In [30]:

```
plt.figure(figsize=(7, 4))
plt.plot(cost_history)
plt.xlabel("Iteration")
plt.ylabel("Cost J(w,b)")
plt.title("Gradient Descent: Cost vs Iteration")
plt.show()
```

